Innovation Centre for Education

YENEPOYA

(Deemed To Be University)

-

Project – High Level Design
on

Cloud Cost Intelligence Platform for Smart City

Student Name:

Malavika Rajesh

23BCAICD050

Industry Mentor: Mr.Shashank

Guided by: Mrs. Niharika

Table of Contents

1. Introduction

1.1 Scope of the Document
1.2 Intended Audience
1.3 System Overview

2. System Design

2.1 Application Design
2.2 Process Flow
2.3 Information Flow
2.4 Components Design
2.5 Key Design Considerations
2.6 API Catalogue

3. Data Design

3.1 Data Model
3.2 Data Access Mechanism
3.3 Data Retention Policies
3.4 Data Migration

4. Interfaces

4.1 State and Session Management
4.2 Caching

5. Non-Functional Requirements

5.1 Security Aspects
5.2 Performance Aspects

6. References

1. Introduction

1.1 Scope of the Document

This High Level Design (HLD) document describes the overall system architecture and design of the CloudLens Cloud Cost Analytics Dashboard. It covers the application architecture, component interactions, data flows, and non-functional requirements. This document serves as the primary technical reference for developers, architects, and stakeholders involved in the development and review of the system.

1.2 Intended Audience

This document is intended for the following audience:

- Development Team – Engineers implementing the Flask backend and frontend dashboard.
- Technical Architects – For evaluating design decisions and technology choices.
- IBM Industry Mentors – For project guidance and assessment.
- Faculty Guides – For academic evaluation and review.
- QA Team – For understanding the system to design effective test plans.

1.3 System Overview

CloudLens is a web-based cloud cost analytics dashboard built using Python and Flask. It allows users to upload a CSV file containing cloud usage and cost data, and provides interactive visual analytics across multiple dimensions including department spend, regional distribution, service-type breakdown, and daily cost trends.

The application accepts CSV files with the following required columns: Date, Cost_INR, Department, Service_Type, Application, and Region. It processes these using pandas, generates charts via Matplotlib, and renders the results as a single-page dashboard using Jinja2 templates embedded in Flask.

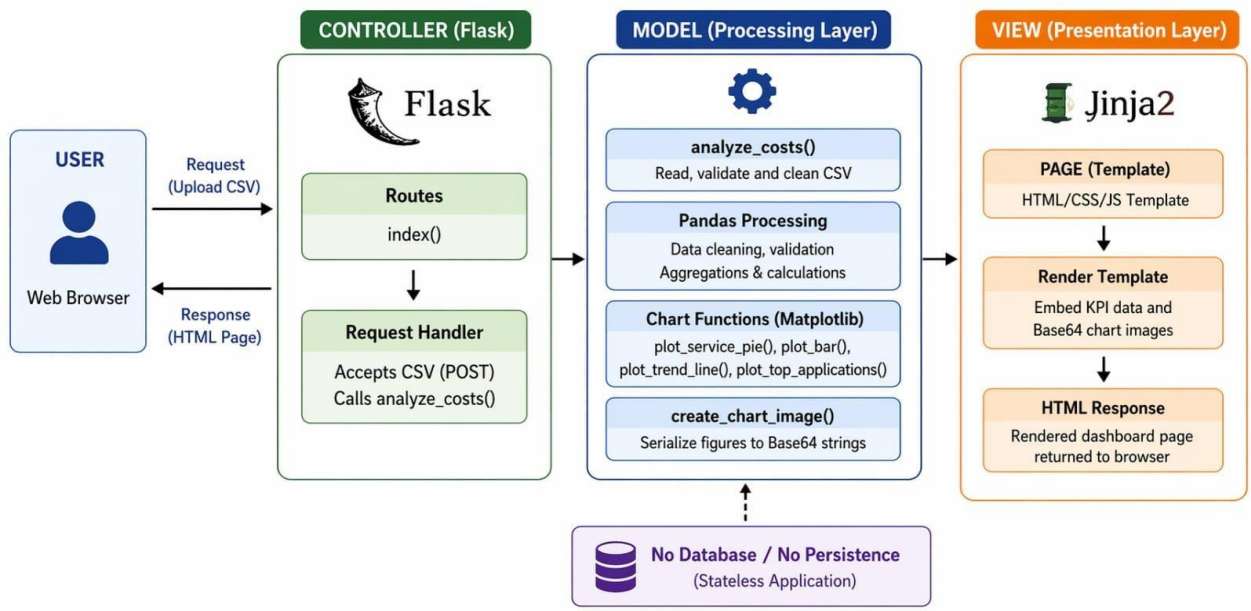
Component	Technology	Role
Web Framework	Flask (Python)	HTTP routing, file upload handling, templating
Data Processing	Pandas	CSV ingestion, groupby aggregations, trend analysis
Visualization	Matplotlib	Chart generation (PNG encoded as Base64)
Frontend	HTML/CSS + Jinja2	Responsive dashboard rendered server-side
Server Encoding	Base64 / BytesIO	Embedding chart images inline in HTML

2. System Design

2.1 Application Design

CloudLens follows a monolithic Model-View-Controller (MVC) pattern within a Flask application. The architecture is intentionally lightweight, targeting local or single-server deployments without external databases or microservices. All state is transient — the application does not persist uploaded files or analysis results between sessions.

The primary design pattern is a request-response cycle: the user submits a CSV via a multipart form POST, the server processes it and returns a fully rendered HTML page with embedded chart images.

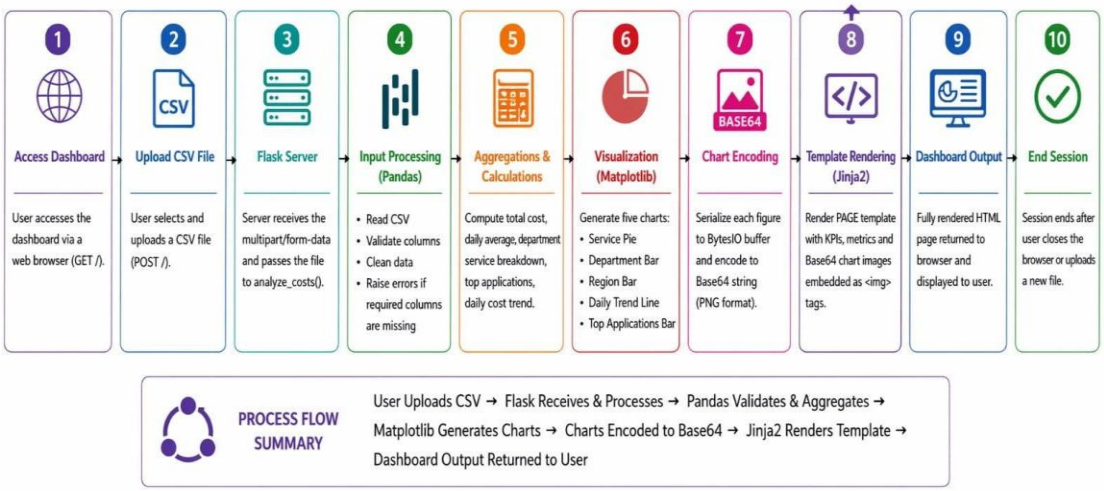


2.2 Process Flow

The following describes the end-to-end process flow of the CloudLens system:

- User accesses the dashboard via a browser (GET /).
- The blank dashboard is rendered with empty KPI and chart placeholders.
- User selects and uploads a CSV file (POST /).
- Flask receives the multipart/form-data and passes the file stream to analyze_costs().
- Pandas reads and validates the CSV; raises errors for missing required columns.
- Aggregations are computed: total cost, daily average, department totals, regional totals, top applications, service breakdown.
- Five chart-generation functions are invoked (plot_service_pie, plot_bar x2, plot_trend_line, plot_top_applications).
- Each Matplotlib figure is serialized to a BytesIO buffer, base64-encoded, and returned as a dictionary.

- Jinja2 renders the PAGE template with KPI metrics and chart image strings embedded in <img src='...'> tags.
- The fully rendered HTML is returned to the browser in a single HTTP response.



2.3 Information Flow

The information flow within CloudLens is unidirectional during a single request lifecycle. Data flows from the CSV file upload through processing and into the rendered view:

Stage	Source	Destination	Data Type
Upload	User Browser	Flask /route	Multipart CSV file stream
Ingestion	Flask FileStorage	Pandas DataFrame	Raw CSV bytes
Processing	DataFrame	Python dicts/Series	Aggregated cost data
Visualization	Pandas Series	Matplotlib Figures	Numeric series + labels
Encoding	Matplotlib Figure	Base64 string	PNG image bytes
Rendering	Jinja2 template + context	HTTP Response	HTML string
Display	HTTP Response	Browser DOM	Rendered HTML page

2.4 Components Design

CloudLens comprises the following major components:

Component	Description	Key Functions
Flask App (app.py)	Main application entry point; defines the single route and orchestrates all processing.	app.route('/'), index()
analyze_costs()	Core data processing module; reads and validates CSV; computes all aggregations.	pd.read_csv, groupby, sum, sort_values
Chart Generator	Five dedicated plotting functions producing dark-theme Matplotlib figures.	plot_service_pie, plot_bar, plot_trend_line, plot_top_applications
create_chart_image()	Serializes Matplotlib figures to base64-encoded PNG strings.	BytesIO, fig.savefig, base64.b64encode
Jinja2 Template (PAGE)	Inline HTML/CSS template defining the entire UI including KPI cards, chart grid, and upload form.	render_template_string()

2.5 Key Design Considerations

- Stateless processing: No server-side file storage. The uploaded CSV is processed entirely in memory and discarded after the response.
- Inline template: The entire HTML/CSS UI is defined as a Python string constant (PAGE), keeping the application a single-file deployment.
- Dark theme consistency: All chart backgrounds use the same CSS color palette (--bg: #08090b, --surface: #0e1117) to match the dashboard theme.
- Error isolation: Exceptions during analysis are caught and surfaced as user-friendly error messages in the UI without crashing the server.

2.6 API Catalogue

CloudLens exposes a single HTTP endpoint:

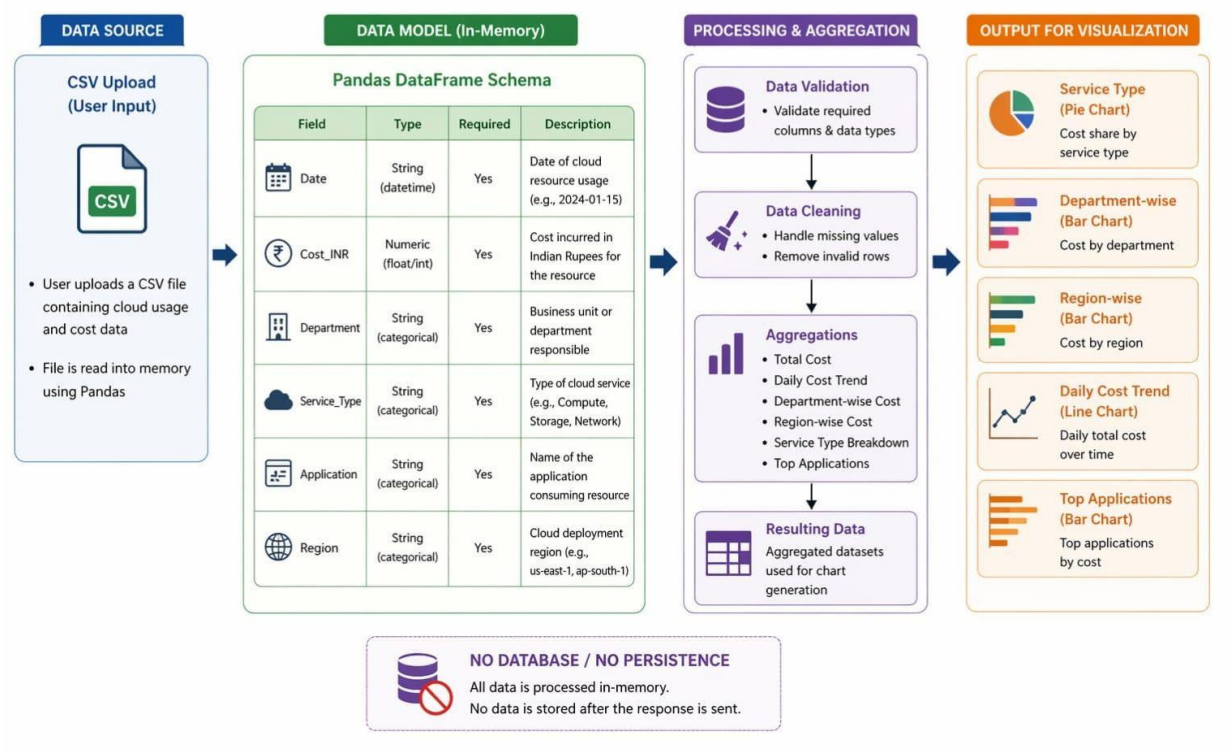
Method	Endpoint	Description	Request Type	Response
GET	/	Renders the blank dashboard with upload form.	None	200 HTML
POST	/	Accepts CSV upload, processes data, returns rendered dashboard with charts and KPIs.	multipart/form-data (field: myfile)	200 HTML

3. Data Design

3.1 Data Model

CloudLens does not use a persistent database. The data model is entirely in-memory and based on a Pandas DataFrame derived from the user-uploaded CSV. The expected schema is as follows:

Field	Type	Required	Description
Date	String (parseable to datetime)	Yes	Date of cloud resource usage (e.g., 2024-01-15)
Cost_INR	Numeric (float/int)	Yes	Cost incurred in Indian Rupees for the resource on that date
Department	String (categorical)	Yes	Business unit or department responsible for the cost
Service_Type	String (categorical)	Yes	Type of cloud service (e.g., Compute, Storage, Network)
Application	String (categorical)	Yes	Name of the application consuming the resource
Region	String (categorical)	Yes	Cloud deployment region (e.g., us-east-1, ap-south-1)



3.2 Data Access Mechanism

Data is accessed exclusively through Pandas operations on an in-memory DataFrame. The access pattern is read-only; no writes occur. The primary access methods are:

- pd.read_csv() — Ingests the uploaded file stream into a DataFrame.
- df.groupby(column)[Cost_INR].sum() — Aggregates cost by categorical dimensions.
- df.groupby('Date')[Cost_INR].sum().sort_index() — Computes the daily cost trend series.
- df.dropna() — Removes rows with missing values in required columns before analysis.

3.3 Data Retention Policies

CloudLens does not retain any uploaded data. The CSV file stream is processed entirely in memory during the request lifecycle and is garbage-collected after the HTTP response is sent. No files are written to disk and no database records are created. Each upload is a fully isolated, stateless operation.

3.4 Data Migration

Not applicable. CloudLens does not maintain a persistent data store; therefore, no data migration strategy is required. Future versions that introduce persistent storage (e.g., a PostgreSQL database for historical trend tracking) would need to define a migration plan at that time.

4. Interfaces

CloudLens presents a single-page web interface. The UI is server-rendered via Jinja2 and returned as a complete HTML document on each POST request. The interface comprises:

- Top Bar: Brand identity, application title, and version badge.
- Page Heading: Dashboard title and description text.
- Upload Card: A styled file input zone with file name display and an Analyse button.
- KPI Row: Four metric cards displaying Total Spend, Average Daily Cost, Top Department, and Top Application.
- Chart Grid: A responsive 12-column CSS grid containing five charts: Daily Cost Trend (wide), Service Breakdown (pie), By Department (bar), Regional Distribution (bar), and Top Applications (bar).

All chart images are delivered inline as Base64-encoded PNG data URIs, eliminating any external asset requests. The frontend uses Google Fonts (DM Mono, Sora) loaded from an external CDN.

4.1 State and Session Management

CloudLens is entirely stateless at the server side. There are no user sessions, no cookies, and no server-side caching of results. Each POST request is a fully independent processing cycle. Client-side state is minimal — only the file name label is updated via a small inline JavaScript function (updateName) when a file is selected.

4.2 Caching

No caching layer is implemented in the current version. Since the application is stateless and all outputs are dynamically generated per upload, caching is not applicable at this stage. Future enhancements could introduce HTTP response caching headers or a Redis-based cache keyed on a hash of the uploaded CSV content to avoid reprocessing identical files.

5. Non-Functional Requirements

5.1 Security Aspects

Concern	Current Approach	Recommendation
File Upload Safety	Flask's request.files prevents path traversal. No file is written to disk.	Add server-side MIME type validation to reject non-CSV files.
Input Validation	pd.to_numeric and pd.to_datetime with errors='coerce' sanitize numeric and date inputs.	Add explicit column whitelist checks before processing.
Error Disclosure	Exception messages from pandas are surfaced directly in the UI.	Sanitize error messages in production to avoid stack trace exposure.
No Authentication	The app is currently open — no login or API key required.	Add HTTP Basic Auth or OAuth2 for production deployments.
Dependency Security	Flask, Pandas, and Matplotlib are widely used open-source libraries.	Pin dependency versions in requirements.txt and run regular vulnerability scans.

5.2 Performance Aspects

Aspect	Current Behavior	Target / Recommendation
CSV Size Limit	No explicit limit enforced by the app. Flask defaults to no limit.	Set MAX_CONTENT_LENGTH = 10 * 1024 * 1024 (10 MB) in Flask config.
Chart Generation Time	5 Matplotlib figures are generated synchronously per request.	For large CSVs (>10k rows), consider async chart generation or caching.
Concurrency	Flask dev server is single-threaded.	Deploy with Gunicorn (multi-worker) for production use.
Memory Usage	Entire CSV is loaded into RAM as a DataFrame.	For very large files, consider chunk-based processing via pd.read_csv(chunksize=).
Response Size	5 Base64 PNG images may add 0.5–2 MB to the HTML response.	Compress response with Flask-Compress (gzip) middleware.

6. References

#	Reference	URL / Source
1	Flask Documentation	https://flask.palletsprojects.com/
2	Pandas User Guide	https://pandas.pydata.org/docs/user_guide/
3	Matplotlib Documentation	https://matplotlib.org/stable/contents.html
4	Jinja2 Template Designer Docs	https://jinja.palletsprojects.com/en/3.x/templates/
6	Python base64 module	https://docs.python.org/3/library/base64.html